Project 1

Library

Here are the packages that I have installed for this project.

In [1]:
```
##import packages here

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# knn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error

# naïve bayes
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix
```

Importing the dataset I will be working with for this project.

In [2]:
```
df = pd.read_csv("nba_2022_2023.csv", na_values = ["", "N/A", "-"])
print(df.head())
```

```
             Player Pos  Age   Tm   G  GS    MP   FG   FGA    FG%  ...    F
T%  \
0  Precious Achiuwa   C   23  TOR  55  12  20.7  3.6   7.3  0.485  ...  0.70
2
1      Steven Adams   C   29  MEM  42  42  27.0  3.7   6.3  0.597  ...  0.36
4
2       Bam Adebayo   C   25  MIA  75  75  34.6  8.0  14.9  0.540  ...  0.80
6
3      Ochai Agbaji  SG   22  UTA  59  22  20.5  2.8   6.5  0.427  ...  0.81
2
4      Santi Aldama  PF   22  MEM  77  20  21.8  3.2   6.8  0.470  ...  0.75
0

   ORB  DRB   TRB  AST  STL  BLK  TOV   PF   PTS
0  1.8  4.1   6.0  0.9  0.6  0.5  1.1  1.9   9.2
1  5.1  6.5  11.5  2.3  0.9  1.1  1.9  2.3   8.6
2  2.5  6.7   9.2  3.2  1.2  0.8  2.5  2.8  20.4
3  0.7  1.3   2.1  1.1  0.3  0.3  0.7  1.7   7.9
4  1.1  3.7   4.8  1.3  0.6  0.6  0.8  1.9   9.0

[5 rows x 29 columns]
```

NBA Players Stats 2022-2023 Season

Downloaded this dataset from Kaggle.* There are 679 rows and 29 columns

*Citation: Chung, B. W. (2023, September 28). NBA Player stats dataset for the 2022-2023. Kaggle. https://www.kaggle.com/datasets/bryanchungweather/nba-players-data-2022-2023?select=nba_2022_2023.csv

In [3]:
```
df
```

Out[3]:

	Player	Pos	Age	Tm	G	GS	MP	FG	FGA	FG%	...	FT%	ORB	DRE
0	Precious Achiuwa	C	23	TOR	55	12	20.7	3.6	7.3	0.485	...	0.702	1.8	4.1
1	Steven Adams	C	29	MEM	42	42	27.0	3.7	6.3	0.597	...	0.364	5.1	6.5
2	Bam Adebayo	C	25	MIA	75	75	34.6	8.0	14.9	0.540	...	0.806	2.5	6.7
3	Ochai Agbaji	SG	22	UTA	59	22	20.5	2.8	6.5	0.427	...	0.812	0.7	1.3
4	Santi Aldama	PF	22	MEM	77	20	21.8	3.2	6.8	0.470	...	0.750	1.1	3.7
...	...	...	...	...	...	...	...	...	...	...	...	...	...	..
674	Thaddeus Young	PF	34	TOR	54	9	14.7	2.0	3.7	0.545	...	0.692	1.3	1.8
675	Trae Young	PG	24	ATL	73	73	34.8	8.2	19.0	0.429	...	0.886	0.8	2.2
676	Omer Yurtseven	C	24	MIA	9	0	9.2	1.8	3.0	0.593	...	0.833	0.9	1.7
677	Cody Zeller	C	30	MIA	15	2	14.5	2.5	3.9	0.627	...	0.686	1.7	2.6
678	Ivica Zubac	C	25	LAC	76	76	28.6	4.3	6.8	0.634	...	0.697	3.1	6.8

679 rows × 29 columns

What each column stands for:

- Player: Player's Name
- Pos: Position
- Age: Player's Age
- Tm: Team
- G: Games Played
- MP: Minutes played per game
- FG: Field goals per game
- FGA: Field goal attempts per game
- FG%: Field goal percentage
- 3P: 3-point field goals per game
- 3PA: 3-point field goal attempts per game
- 3P%: 3-point field goal percentage
- 2P: 2-point field goals per game
- 2PA: 2-point field goal attempts per game

- 2P%: 2-point field goal percentage
- eFG%: Effective field goal percentage
- FT: Free throws per game
- FTA: Free throw attempts per game
- FT%: Free throw percentage
- ORB: Offensive rebounds per game
- DRB: Defensive rebounds per game
- TRB: Total rebounds per game
- AST: Assists per game
- STL: Steals per game
- BLK: Blocks per game
- TOV: Turnovers per game
- PF: Personal fouls per game
- PTS: Points per game

Data Cleaning

Cleaning this data requires adding 0s to all of the missing data. Out of the almost 20,000 cells, there were only 74 cells that had missing information.

In [4]:
```
df.fillna(0, inplace = True)
```

There were duplicates of certain players due to trades during the season. There were 70 players who got traded at one point during the season. Before the data cleaning, each person was represented 3 or more plus time depending on how many teams they were on, one row inlcluding the total amount of time. I've identified the duplicate players, kept the games where they only played the Total (TOT) games in he team column to represent their time that season. Now, there are 539 rows instead of 679.

In [5]:
```
tot = df[df["Tm"] == "TOT"]
#print(tot["Player"].unique())
print(tot["Player"].count())
print(df.iloc[3:9, 0:5])
```

```
70
                     Player Pos  Age   Tm   G
3              Ochai Agbaji  SG   22  UTA  59
4              Santi Aldama  PF   22  MEM  77
5  Nickeil Alexander-Walker  SG   24  TOT  59
6  Nickeil Alexander-Walker  SG   24  UTA  36
7  Nickeil Alexander-Walker  SG   24  MIN  23
8             Grayson Allen  SG   27  MIL  72
```

In [6]:
```
#tradedplayers = df[df.duplicated(subset=["Player"], keep=False)]["Player"].
#cleandf = df[(df["Tm"] == "TOT") | (~df["Player"].isin(tradedplayers))]
#cleandf = cleandf.drop_duplicates(subset=["Player"], keep="first").reset_in

tradedplayers = df[df.duplicated(subset = ["Player"], keep = False)]["Player
```

```
cleandf = df[(df["Tm"] == "TOT") | (~df["Player"].isin(tradedplayers))]
cleandf = cleandf.drop_duplicates(subset=["Player"], keep="first").reset_inc

print(cleandf.iloc[3:9, 0:5])
```

```
                     Player Pos  Age   Tm   G
3              Ochai Agbaji  SG   22  UTA  59
4              Santi Aldama  PF   22  MEM  77
5  Nickeil Alexander-Walker  SG   24  TOT  59
6             Grayson Allen  SG   27  MIL  72
7             Jarrett Allen   C   24  CLE  68
8             Jose Alvarado  PG   24  NOP  61
```

Finally, I made a new data frame with only the columns that I will be using during this project.

In [7]:
```
selectedcolumns = ["Player", "Age", "G", "MP", "PTS", "AST", "TRB"]
newdf = cleandf[selectedcolumns]
print(newdf)
```

```
               Player  Age   G    MP   PTS   AST   TRB
0    Precious Achiuwa   23  55  20.7   9.2   0.9   6.0
1        Steven Adams   29  42  27.0   8.6   2.3  11.5
2         Bam Adebayo   25  75  34.6  20.4   3.2   9.2
3        Ochai Agbaji   22  59  20.5   7.9   1.1   2.1
4        Santi Aldama   22  77  21.8   9.0   1.3   4.8
..                ...  ...  ..   ...   ...   ...   ...
534    Thaddeus Young   34  54  14.7   4.4   1.4   3.1
535        Trae Young   24  73  34.8  26.2  10.2   3.0
536    Omer Yurtseven   24   9   9.2   4.4   0.2   2.6
537       Cody Zeller   30  15  14.5   6.5   0.7   4.3
538       Ivica Zubac   25  76  28.6  10.8   1.0   9.9

[539 rows x 7 columns]
```

Goal for this project

My goal for this project is to investigate how points correlate with overall player performance by analyzing key metrics including rebounds, assists, and other performance indicators. By grouping players based on their scoring output, I aim to identify relationships between points and other various aspects of basketball performance.

Exploratory Data Analysis

Players in Each Age Group

In this next section, I combined similar ages and grouped them together to represent an age group. There are three ages in each age group and labeled them their respective ages. Then, I made a pie chart displaying how many ages that are in each age group.

In [8]:
```
newdf = cleandf[["Player", "Age", "G", "MP", "PTS", "AST", "TRB"]].copy()

sorting = [
    newdf["Age"].between(19, 21),
    newdf["Age"].between(22, 24),
    newdf["Age"].between(25, 27),
    newdf["Age"].between(28, 30),
    newdf["Age"].between(31, 33),
    newdf["Age"].between(34, 36),
    newdf["Age"].between(37, 39),
    newdf["Age"].between(40, 42)
]
labels = ["Late Teens", "Early 20s", "Mid 20s", "Late 20s",
          "Early 30s", "Mid 30s", "Late 30s", "Early 40s"]
newdf["AG"] = np.select(sorting, labels, default = "Unknown")
#newdf[["Player", "Age", "AG"]].head()
newdf.head()
```

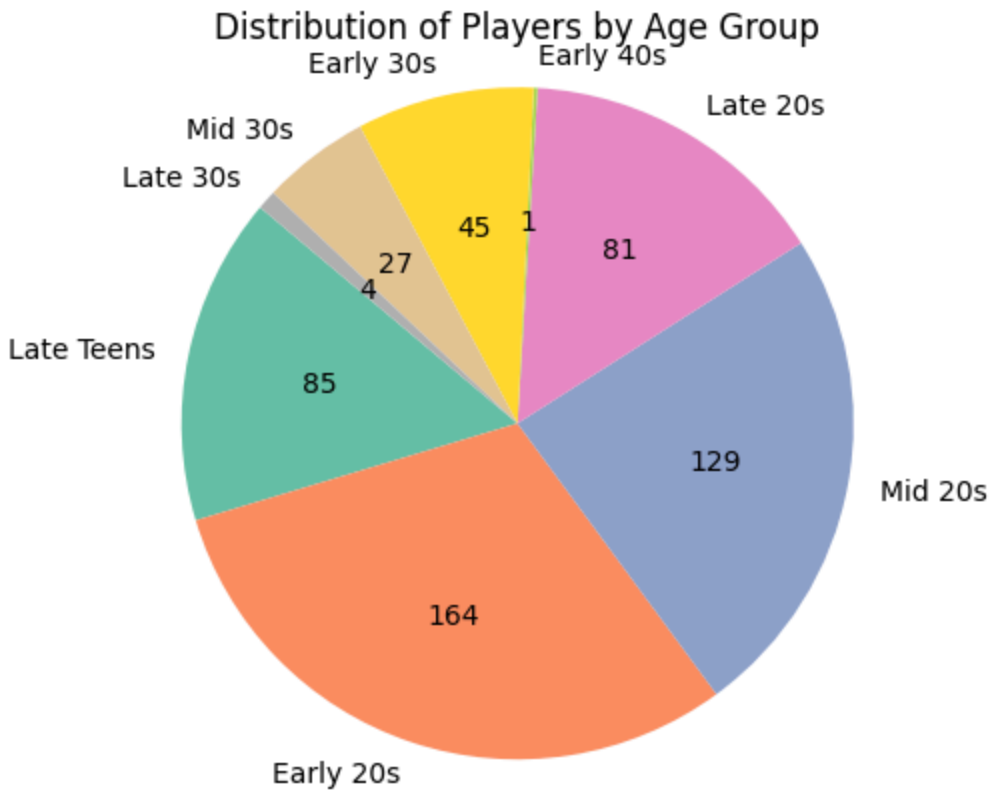
Out[8]:

	Player	Age	G	MP	PTS	AST	TRB	AG
0	Precious Achiuwa	23	55	20.7	9.2	0.9	6.0	Early 20s
1	Steven Adams	29	42	27.0	8.6	2.3	11.5	Late 20s
2	Bam Adebayo	25	75	34.6	20.4	3.2	9.2	Mid 20s
3	Ochai Agbaji	22	59	20.5	7.9	1.1	2.1	Early 20s
4	Santi Aldama	22	77	21.8	9.0	1.3	4.8	Early 20s

In [9]:
```
ageorder = ["Late Teens", "Early 20s", "Mid 20s", "Late 20s", "Early 40s",
          "Early 30s", "Mid 30s", "Late 30s"]
countagegroup = newdf["AG"].value_counts()
countagegroup = countagegroup.reindex(ageorder)
newdf["AgeGroupCount"] = newdf["AG"].map(countagegroup)

def func(pct, allvalues):
    absolute = int(pct / 100.*np.sum(allvalues))
    return f"{absolute}"

plt.pie(countagegroup, labels = countagegroup.index, autopct= lambda pct: fu
        startangle = 140, colors = sns.color_palette("Set2", len(countagegro
plt.axis('equal')
plt.title("Distribution of Players by Age Group")
plt.show()
```



Most of the players are in the age range 19-30 with 459 players (85% of the league), while there are only 77 players older than 30 (15% of the league).

- Late Teens (19-21) - 85 players
- Early 20s (22-24) - 164 players
- Mid 20s (25-27) - 129 players
- Late 20s (28-30) - 81 players
- Early 30s (31-33) - 45 players
- Mid 30s (34-36) - 27 players
- Late 30s (37-39) - 4 players
- Early 40s (40-42) - 1 players

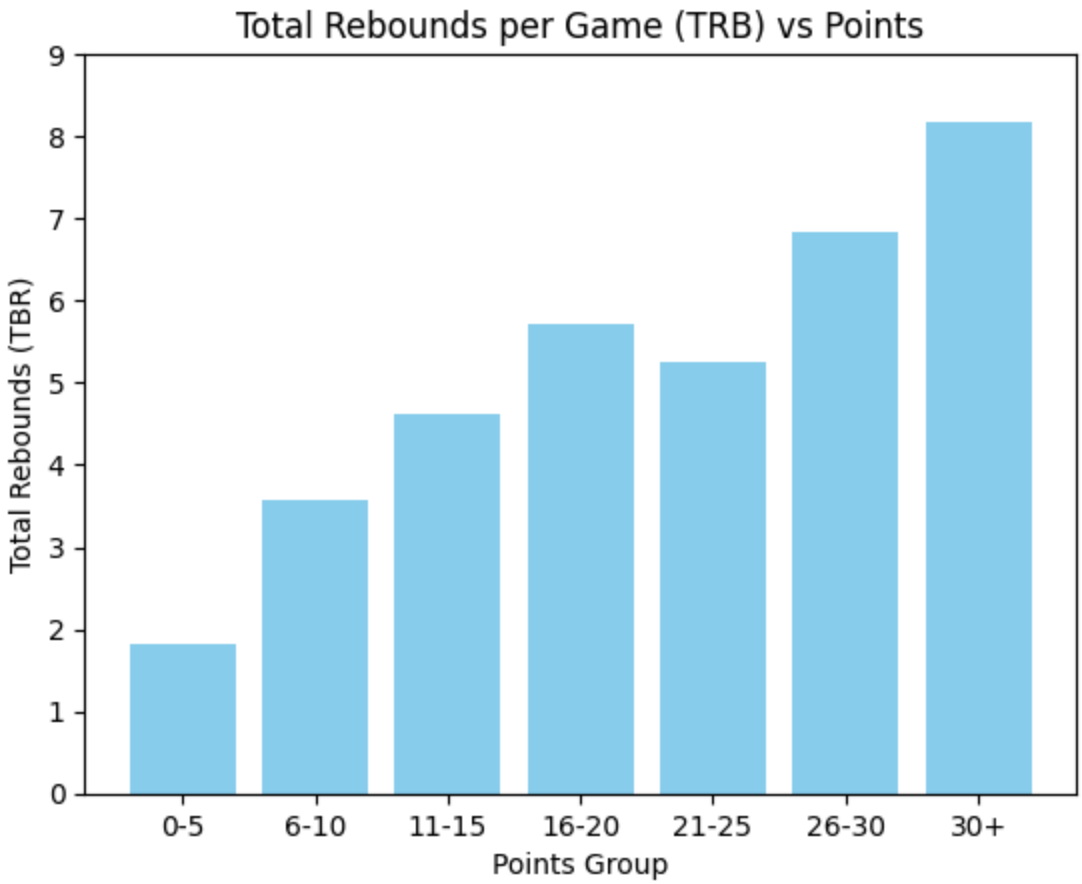
Total Rebounds per Game vs Points

In [10]:
```
bins = [0, 5, 10, 15, 20, 25, 30, 35]
labels = ["0-5", "6-10", "11-15", "16-20", "21-25", "26-30", "30+"]
newdf["PTS_group"] = pd.cut(newdf["PTS"], bins=bins, labels=labels, include_
avgppg = newdf.groupby("PTS_group", observed=False)["PTS"].mean().reset_inde

total_rebounds = newdf.groupby("PTS_group", observed=True)["TRB"].mean().res

#plt.figure(figsize=(10, 6))
plt.ylim(0,9)
plt.bar(total_rebounds["PTS_group"], total_rebounds["TRB"], color='skyblue')
plt.title("Total Rebounds per Game (TRB) vs Points")
plt.xlabel("Points Group")
```

```
plt.ylabel("Total Rebounds (TBR)")
plt.show()
```



In the graph above, it represents the average of the total rebounds per game by point group. This shows that players who average more points per game get more total rebounds per game.

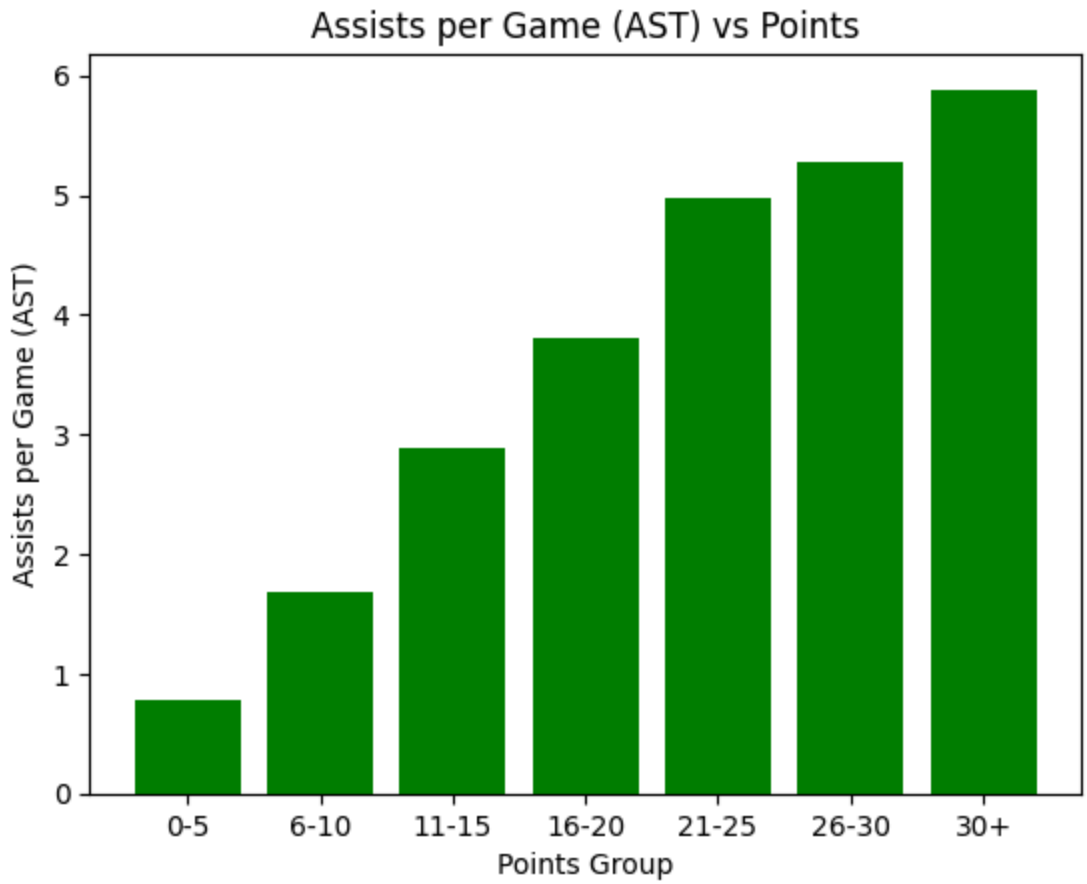
- 0-5 point group: 1.8 rebounds
- 6-10 point group: 3.5 rebounds
- 11-15 point group: 4.5 rebounds
- 16-20 point group: 5.8 rebounds
- 21-25 point group: 5.2 rebounds
- 26-30 point group: 6.5 rebounds
- 30-35 point group: 8.1 rebounds

Assists per Game vs Points

In [11]:
```
avg_apg = newdf.groupby("PTS_group", observed=True)["AST"].mean().reset_inde

#plt.figure(figsize=(10, 6))
plt.bar(avg_apg["PTS_group"], avg_apg["AST"], color = "green")
plt.title("Assists per Game (AST) vs Points")
plt.xlabel("Points Group")
```

```
plt.ylabel("Assists per Game (AST)")
plt.show()
```



The graph above shows the amount of assists per game by the point group. The correlation of this graph shows that when players have more points in a game, those players get more assists per game.

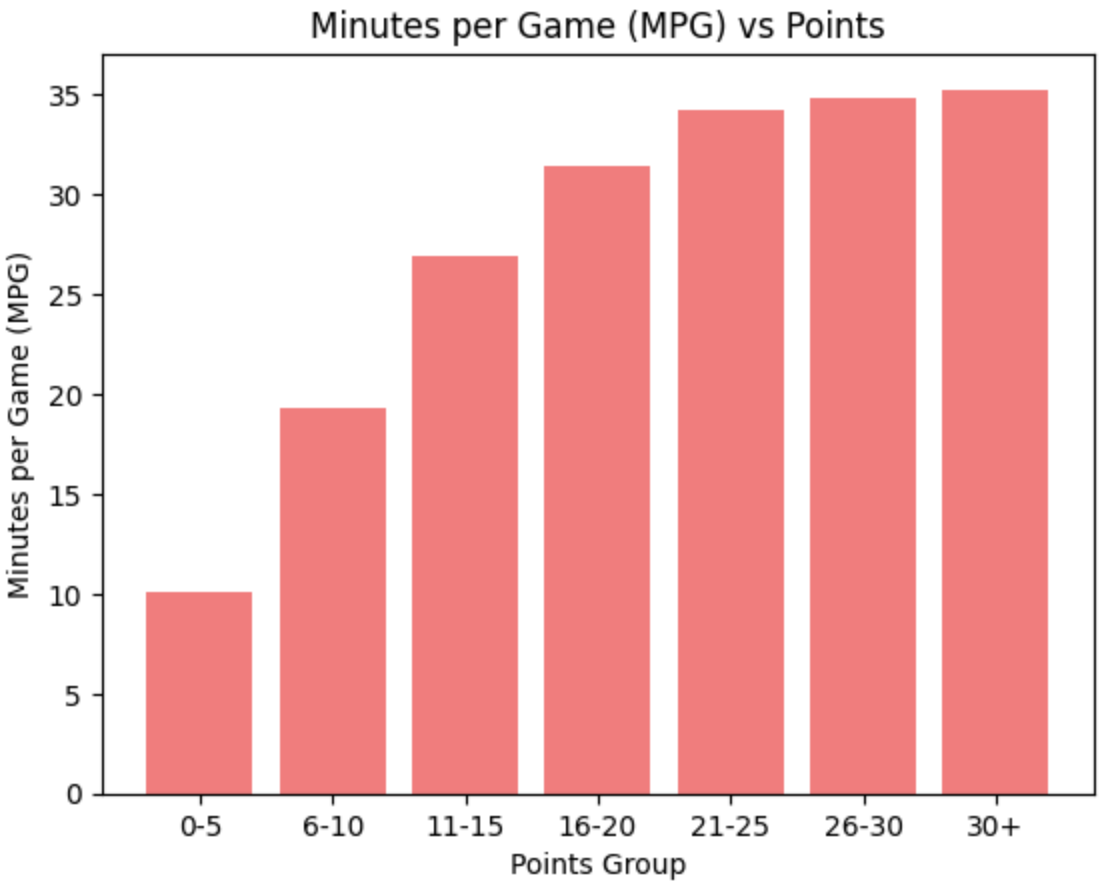
- 0-5 point group: 0.7 assists
- 6-10 point group: 1.6 assists
- 11-15 point group: 2.8 assists
- 16-20 point group: 4.8 assists
- 21-25 point group: 5.0 assists
- 26-30 point group: 5.4 assists
- 30-35 point group: 5.8 assists

Minutes Per Game vs Points

In [12]:
```
avg_mpg = newdf.groupby("PTS_group", observed=True)["MP"].mean().reset_index

#plt.figure(figsize=(10, 6))
plt.bar(avg_mpg["PTS_group"], avg_mpg["MP"], color = "lightcoral")
plt.title("Minutes per Game (MPG) vs Points")
plt.xlabel("Points Group")
```

```
plt.ylabel("Minutes per Game (MPG)")
plt.show()
```



The graph above shows the amount of assists per game by the point group. The correlation of this graph shows that when players get more minutes per game, those players average more points per game. The point groups 20-25, 26-30, and 30-35 have almost identical minutes to points ratio.

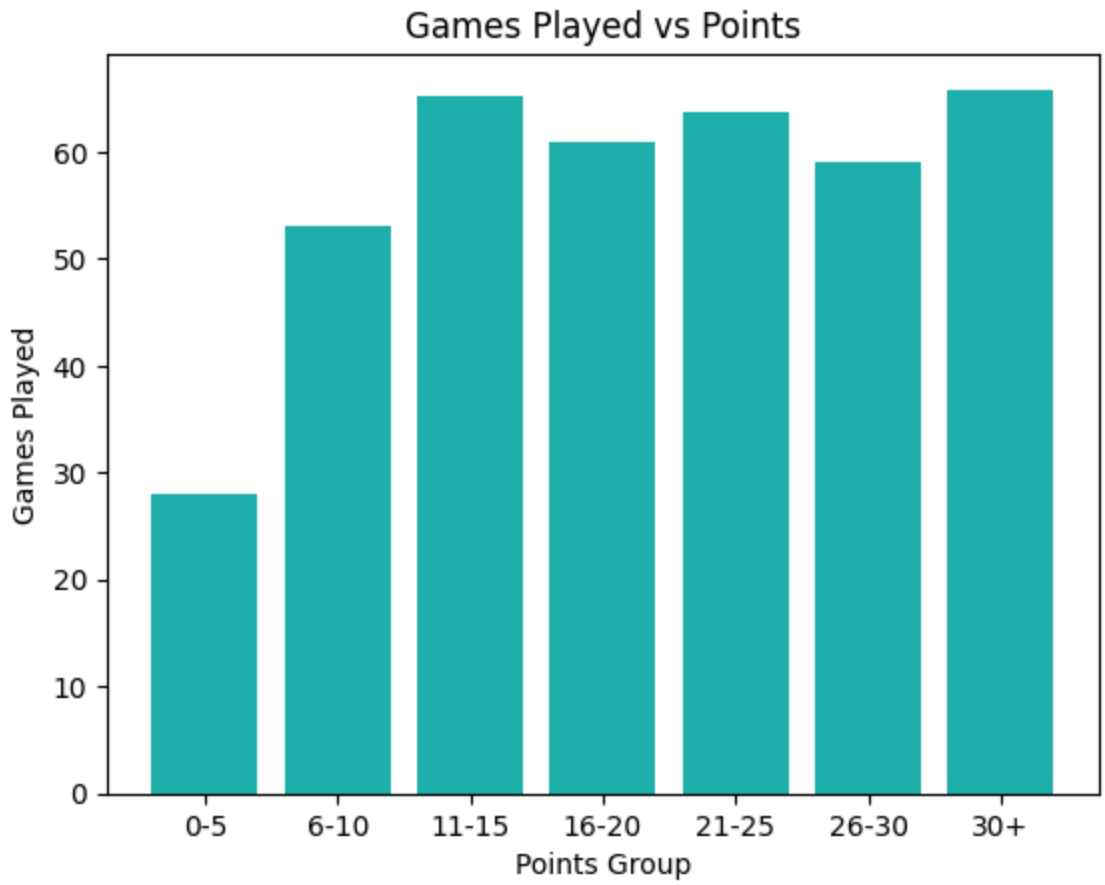
- 0-5 point group: 10 points
- 6-10 point group: 18 points
- 11-15 point group: 27 points
- 16-20 point group: 30 points
- 21-25 point group: 33 points
- 26-30 point group: 34 points
- 30-35 point group: 35 points

Games Played vs Points

In [13]:
```
avg_games_played = newdf.groupby("PTS_group", observed=True)["G"].mean().res

#plt.figure(figsize=(10, 6))
plt.bar(avg_games_played["PTS_group"], avg_games_played["G"], color = "light
plt.title("Games Played vs Points")
plt.xlabel("Points Group")
```

```
plt.ylabel("Games Played")
plt.show()
```



The graph displays the amount of games played by the point group. This graph shows that there is no correlation between games played and points.

- 0-5 point group: 28 games played
- 6-10 point group: 53 games played
- 11-15 point group: 65 games played
- 16-20 point group: 63 games played
- 21-25 point group: 62 games played
- 26-30 point group: 60 games played
- 30-35 point group: 65 games played

Training and Testing Sets with Naïve Bayes and KNN

For both Naïve Bayes and KNN models, I selected the "PTS" column as a key feature for this prediction. Since my goal is to analyze player performance, points is a crucial metric as it correlates with a players overall performance. By using the same points column across both models, we can observe how both models predicts outcomes.

Naïve Bayes

In [14]:
```
X = newdf.drop(columns=["PTS"])
X = X.select_dtypes(include = [np.number])
y = newdf["PTS"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

# convert PTS into bins
y_train_class = pd.cut(y_train, bins = [0, 10, 20, 30], labels = ["Low", "Me
y_test_class = pd.cut(y_test, bins = [0, 10, 20, 30], labels = ["Low", "Medi

# handle missing values
y_train_class = y_train_class.dropna()
y_test_class = y_test_class.dropna()

# ensuring X_train and X_test match y_train_class and y_test_class
X_train = X_train.loc[y_train_class.index]
X_test = X_test.loc[y_test_class.index]

# train model
nb = GaussianNB()
nb.fit(X_train, y_train_class)

# prediction
y_pred_class = nb.predict(X_test)

# model performance
accuracy = accuracy_score(y_test_class, y_pred_class)

print(f"Naïve Bayes Accuracy: {accuracy:.4f}")
print(classification_report(y_test_class, y_pred_class))
```

```
Naïve Bayes Accuracy: 0.8585
              precision    recall  f1-score   support

        High       0.67      1.00      0.80         6
         Low       0.96      0.89      0.92        79
      Medium       0.62      0.71      0.67        21

    accuracy                           0.86       106
   macro avg       0.75      0.87      0.80       106
weighted avg       0.88      0.86      0.86       106
```

From the code above, I converted PTS into low, medium, and high categories.

- low: 0-10 points
- medium: 10-20 points
- high: 20-30 points

I then handled the missing values and trained the model. The model's accuracy score was 85.85%, which is an average accuracy rate.

Precision: The precision column measures the true positives divided by the true positives plus the false positives. In the high and medium categories, they are being misclassified in different categories, while the low category is almost always correct.

- High = 67% precision
- Low = 96% precision
- Medium = 62% precision

Recall: The recall column measured the true positives divided by the true positives plus the false negatives. In the high category, there are no false negatives. The low category is doing a good job but could use some improvements. The medium category is almost missing 30% of the points which is not so good.

- High = 100% recall
- Low = 89% recall
- Medium = 71% recall

F1-score: The f1-score column takes the average of the precision and recall columns. Low has the highest f1-score and is pretty accurate predicting low points. High has a decent score with the concern of false positives from the prediciton category. Medium is the weakest category and the model stuggles to predict in this point class.

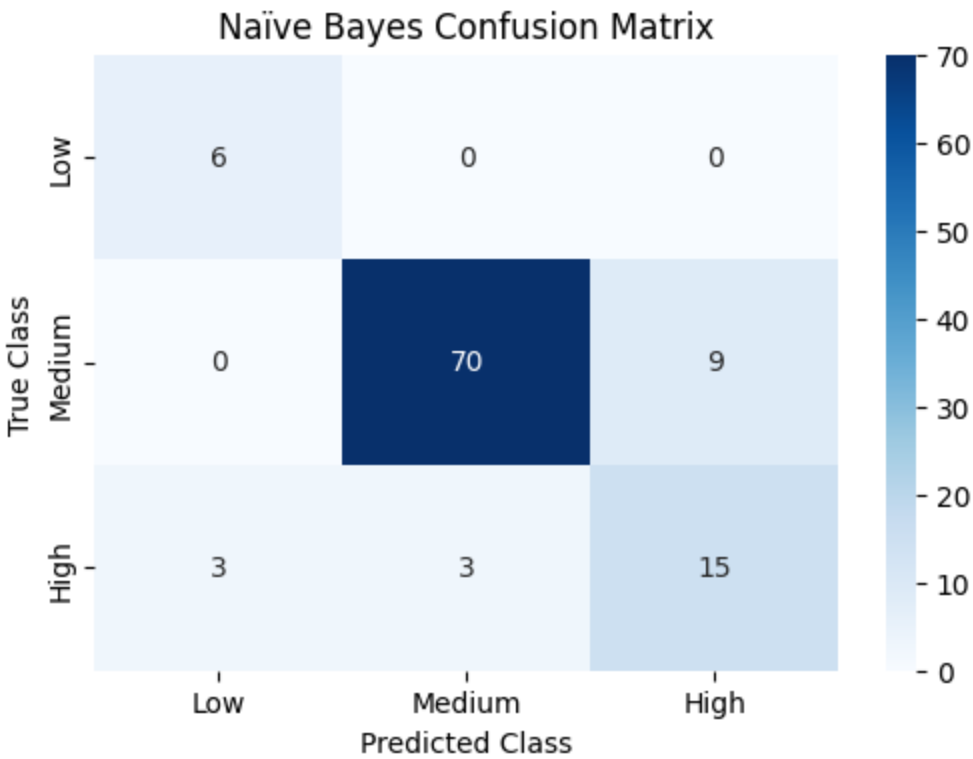
- High = 80% f1-score
- Low = 92% f1-score
- Medium = 67% f1-score

Overall, the naïve bayes model performs pretty well, but there is room for improvements.

Naïve Bayes Graph

In [15]:
```
# confusion matrix
cm = confusion_matrix(y_test_class, y_pred_class)

plt.figure(figsize = (6, 4))
sns.heatmap(cm, annot = True, fmt = "d", cmap = "Blues",
            xticklabels=["Low", "Medium", "High"], yticklabels = ["Low", "Me
plt.title("Naïve Bayes Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("True Class")
plt.show()
```



In the confusion matrix above, it shows the performance of the model by comparing the predicted class by the actual class.

- 6 lows instances were correctly classified with no errors
- 15 high instances were correctly classified with 6 instances misclassified -> 3 classified as low and 3 classified as medium
- 70 mediums instances were correctly classified with 9 instances were misclassified as high

The model shows that the classifier performs well as most of the predictions fall along the diagonal line (86%) while 14% were misclassified.

KNN

In [16]:
```
#drop non-numeric columns
X = newdf.drop(columns = ["PTS"])
X = X.select_dtypes(include = [np.number])
y = newdf["PTS"]

# train / test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, r

X_train = X_train.reset_index(drop = True)
y_train = y_train.reset_index(drop = True)

# rescale data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)

# initialize knn regressor
knn = KNeighborsRegressor()

# hyperparameter tuning
paramgrid = {"n_neighbors": [3, 5, 7, 10, 15]}
gridsearch = GridSearchCV(knn, paramgrid, cv = 5, scoring = "neg_mean_square
gridsearch.fit(X_train_scaled, y_train)

# bestk value
bestk = gridsearch.best_params_["n_neighbors"]
print(f"Best k: {bestk}")

# train knn with bestk
knnbest = KNeighborsRegressor(n_neighbors = bestk)
knnbest.fit(X_train_scaled, y_train)

# prediction
yprediction = knnbest.predict(X_test_scaled)

# model performance
mae = mean_absolute_error(y_test, yprediction)
rmse = np.sqrt(mean_squared_error(y_test, yprediction))

print(f"k-NN Regression MAE (PTS): {mae:.4f}")
print(f"k-NN Regression RMSE (PTS): {rmse:.4f}")
```

```
Best k: 7
k-NN Regression MAE (PTS): 2.1538
k-NN Regression RMSE (PTS): 3.0822
```

The results of the knn model shows that the optimal number of neighbors (k) is 7. This means that the model predicts the target value by averaging the values of the 7 nearest training points.

The mean average error (MAE) is 2.1538. It measures the average absolute difference between the predicted and actual values. This means that the model's predictions are on average 2.15 points off from the actual values.

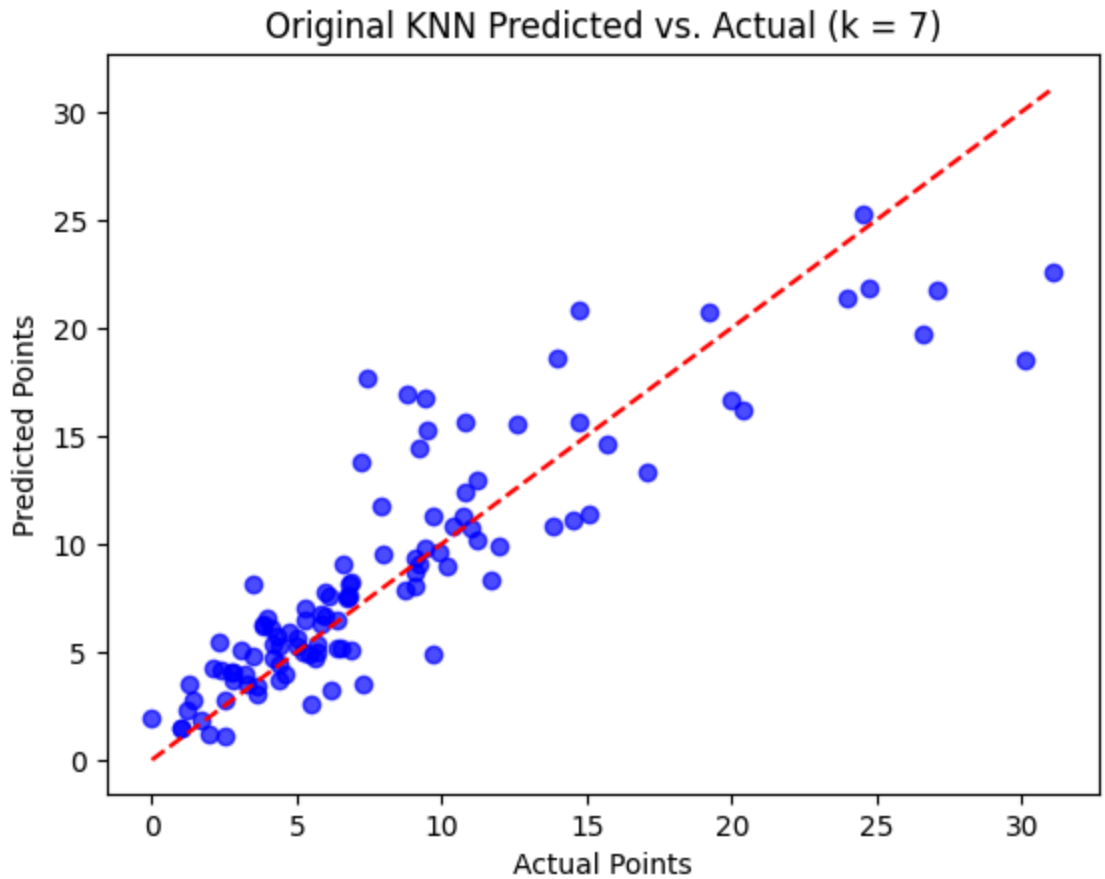
The root mean squared error (RMSE) is 3.0822. It measures how far off the predictions are and has a greater penalty for larget errors. The typical prediction error is around 3 points as the RMSE is more sensitive than the MAE.

Since both the MAE and the RMSE have an average error of 2-3 points, the model is performing well but there are still some error.

KNN Graph

In [17]:
```
plt.scatter(y_test, yprediction, alpha = 0.7, color = "blue")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color =
plt.xlabel("Actual Points")
```

```
plt.ylabel("Predicted Points")
plt.title("Original KNN Predicted vs. Actual (k = 7)")
plt.show()
```



The scatter plot above shows the knn predicted vs the actual values for the knn model. The blue dots represents the predicted values made by the model. The red line represents the perfect model. When the blue dots are closer to the red line, it represents that the model is preforming well and accurate. When the blue dots are farther away from the red line, it represents prediction errors. The blue dots are more scattered as the actual points increase, suggesting that the model struggles predicting higher point values.

Hyperparameter tuning for KNN

In [18]:
```
# hyperparameter grid
paramgrid = {"n_neighbors": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]}

# knn model
knn = KNeighborsRegressor()

# grid search cv
gridsearch = GridSearchCV(knn, paramgrid, cv = 5, scoring = "neg_mean_square
gridsearch.fit(X_train, y_train)

# find bestk
```

```
bestk = gridsearch.best_params_["n_neighbors"]
print(f"Best k: {bestk}")

# train knn with bestk
bestknn = KNeighborsRegressor(n_neighbors=bestk)
bestknn.fit(X_train, y_train)

# prediction
y_pred_knn = bestknn.predict(X_test)

# model performance
mae = mean_absolute_error(y_test, y_pred_knn)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_knn))

print(f"MAE: {mae:.4f}\nRMSE: {rmse:.4f}")
```

```
Best k: 4
MAE: 2.4750
RMSE: 3.4293
```

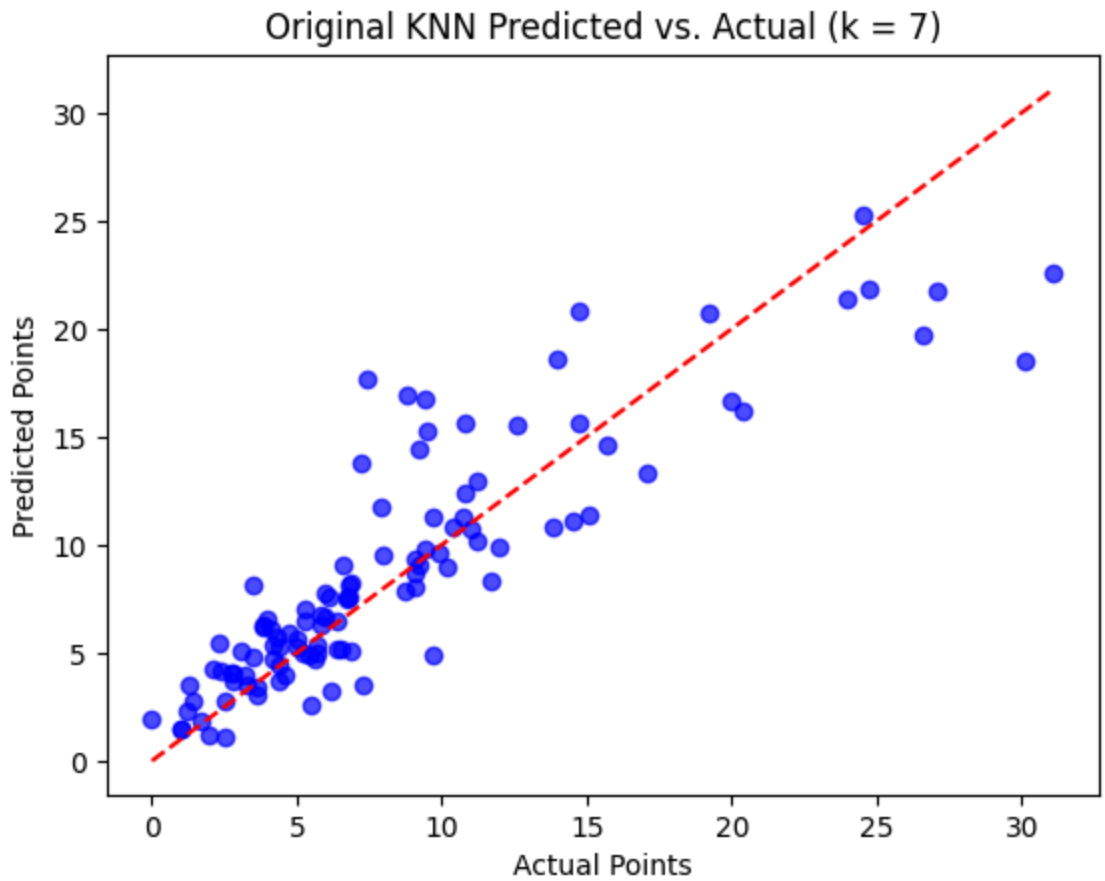
After performing hyperperameter tuning, the best k value is 4.

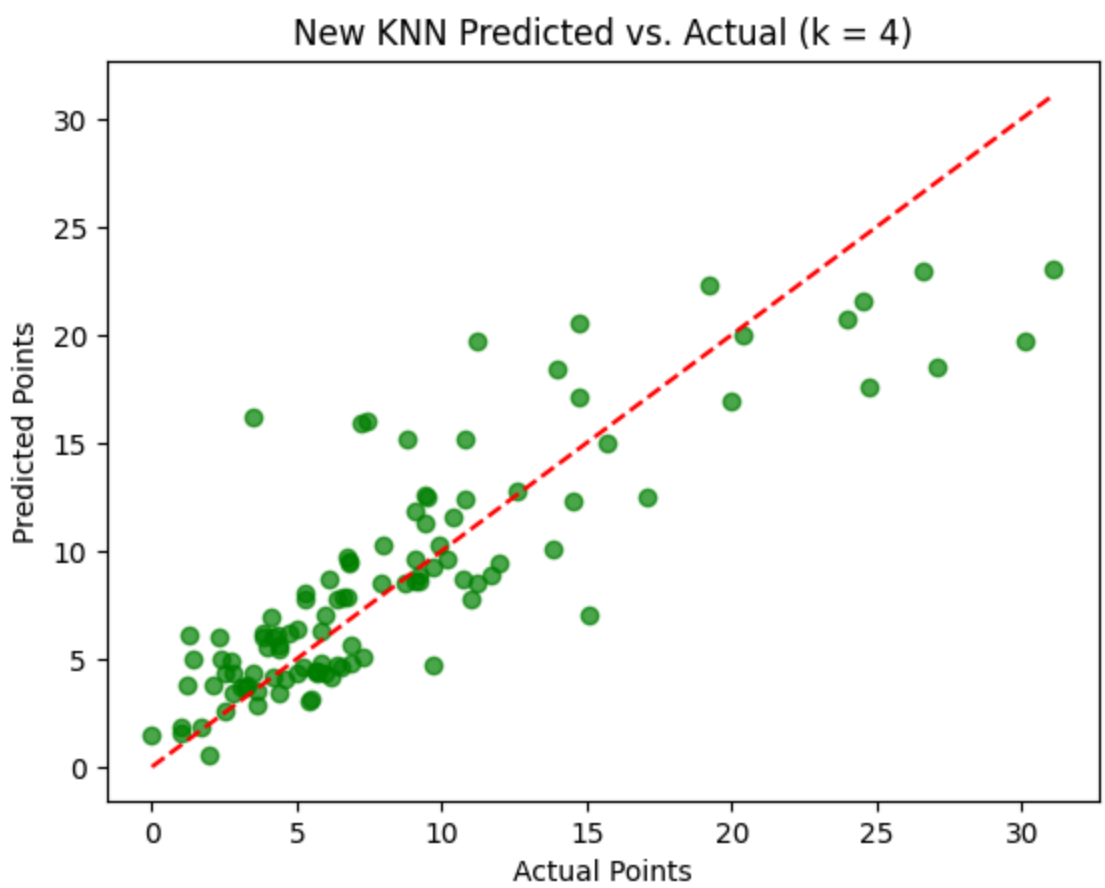
- The MAE is 2.4750
- RMSE score is 3.4293

These results are higher than the original knn model.

In [19]:
```
# origninal knn
plt.scatter(y_test, yprediction, alpha = 0.7, color = "blue")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color =
plt.xlabel("Actual Points")
plt.ylabel("Predicted Points")
plt.title("Original KNN Predicted vs. Actual (k = 7)")
plt.show()

# after hyperparameter knn tuning
plt.scatter(y_test, y_pred_knn, alpha = 0.7, color = "green")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color =
plt.xlabel("Actual Points")
plt.ylabel("Predicted Points")
plt.title("New KNN Predicted vs. Actual (k = 4)")
plt.show()
```





In the original knn model:

- Best k: 7
- MAE: 2.1538
- RMSE: 3.0822

In the new knn model:

- Best k: 4
- MAE: 2.4750
- RMSE: 3.4293

Looking at these two graph, there is not a significant difference between the original model (k = 7) and the new model (k = 4). In the new model, the points are more spread out than The original model. When k = 7 is a little bit more compact and closer to the perfect model.

Looking at the MAE and RMSE scores, the original knn model (k = 7) has the better / lower MAE score and RMSE score, indicating better predictive performance than the newer model (k = 4).

The original knn model (k = 7) is the preferable choice.

Conclusion

Throughout my project, we saw how to clean a dataset, explore the data, and looked testing and training models. The naïve bayes model yielded a 85% accuracy. The low instances had the strongest classifications while medium and high instances were weaker and has some misclassifications. In the knn models, k = 7 had the better predictive performances (MAE = 2.1538, RMSE = 3.0822) compared to when k = 4 (MAE = 2.4750, RMSE = 3.4293), which yielded in higher errors. Overall, the naïve bayes model performed well for classification, and knn with k = 7 was the perferred regression model for prediction points.

In [ ]:

```

```